Spring MD

Spring MD: The Ultimate Potential Function Training and Molecular Dynamics Execution Software

SCX

202671

v1.0 — — SCX — Spring MD

Abstract

Spring MD—— Spring MDSpringYajieM_tArbiter `spring-md train` `spring-md run` `spring-md audit` `spring-md compare`

Spring MD **BORN AUDITED** Spring M_tCercisYajie—— "born audited"MD Spring MDtrajectory Arbiter——CercisYajie

Spring MDNEP/DeepMD/MACE ——

Spring MDAPI LAMMPS/ASEArbiter SCX Spring——

Spring MDArbiter YajieCercisLAMMPSASE

Contents

1

1.1 DFTMD

Molecular Dynamics, MD MD

1 DFT

2 NEPDeepMDACEGAPMTPDFT $V_\theta(\mathbf{R})$

3 MDLAMMPSASE

4 MD——

textbf

- DFTk——
  textit
- RMSE——RMSE ”
  textit”
- MD——
  textit OOD MD
  textit
- ——

MDLAMMPSGROMACSASEVASP ””trust-me——

1.2 SCXMD

SCX[**?**, **?**, **?**, **?**]

(1) $M = 1$——NEPDeepMDACEGAPMTPSCX2

(2) $M > 1$SCX1

(3) SCX3

(4) Spring””Spring Trainer[**?**]

textbf—
Spring MD

1.3 Spring MD

Spring MD
NEP/DeepMD/ACEMDLAMMPS/GROMACS
textbfAudit Middleware—— DFT
Spring MD

1 BORN AUDITED Spring MDM_tCercisYajie "born audited"MD

2 Spring MDArbiter—— CercisYajie

3 NEP/DeepMD/MACE Spring MD`spring-md train`

4 LAMMPS/ASE Spring MDMD——LAMMPSASE

5 DFT

1.4

- **PESPotential Energy Surface** ——$E(\mathbf{R})$
- **Potential Function** PES$V_\theta(\mathbf{R}) \approx E(\mathbf{R})$
- **Trajectory** MD$\{\mathbf{R}(t_1), \ldots, \mathbf{R}(t_T)\}$
- **Audit Report** ArbiterCercisYajieOOD
- **Born Audited** /

1.5

(1) 2——Spring MD

(2) 3`spring-md train`——

(3) 4`spring-md run`——

(4) 5`spring-md audit`——

(5) 6`spring-md compare`——Cercis

(6) 7Arbiter——

(7) 8API——

(8) 9LAMMPS/ASE

(9) 10

(10) 11NEP/DeepMD/MACE

(11) 12——

(12) 13

2 Spring MD

2.1

Spring MDspring-md

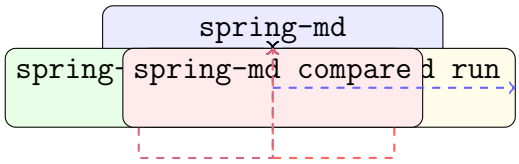


Figure 1: Spring MD trainrunrunaudit traincompare

2.2

Table 1: Spring MD

train	DFTextxyz/npz	+ M_t + Cercis + Yajie	Spring→Yajie→M_t→Cercis
run	+	+ Arbiter	LAMMPS/ASE +
audit	+	Arbiter	Yajie + OOD
compare	+	Cercis	

2.3

Spring MDMD
textbf

1. train Spring$V_{\text{consensus}}$ M_tCercisS_{Cercis}Yajies$_{\text{Yajie}}$ LyapunovΨ
2. run → LAMMPS/ASEMD → kArbiterCercisYajie → " + "
3. audit Spring MDspring-md audit YajieOODMDLAMMPSGROMACS
4. compare spring-md compare Cercis

Remark 2.1. Spring MD
textbfAudit Non-Bypassability spring-md runMD
textitArbiter—— ""flag LAMMPSASE——Spring MD

3 spring-md train

3.1

Listing 1: spring-md train

```
spring-md train [OPTIONS] <INPUT>

INPUT:
  <data.xyz>          DFT (extended XYZ)
  <data.npz>          NPZ ({R, E, F, Z})

OPTIONS:
  --config <FILE>     Spring (YAML)
  --output <DIR>       (: ./spring_output/)
  --arch <ARCH>       : nep | deepmd | ace | mace (: nep)
  --M-min <INT>        (: 3)
  --M-max <INT>        (: 32)
  --epochs <INT>       (: 1000)
  --batch-size <INT>   (: 32)
  --lr <FLOAT>         (: 1e-3)
  --noise-threshold <FLOAT> Yajie (: 0.05)
  --device <STR>      : cuda | cpu (: cuda)
  --seed <INT>         (: 42)

OUTPUT ( <DIR>/ ):
  potential.pt         (PyTorch)
  potential.yaml       (YAML)
  audit_report.json   Cercis + Yajie + Mt + Lyapunov
  training_log.csv     (loss, consensus, lyap per epoch)
  checkpoints/
```

3.2

spring-md trainSPRING

Figure 2: `spring-md train`(3)(2)

1Data Ingestion

DFT

- **extended XYZ**
- **NPZ** NumPyR$N_{\text{data}} \times N_{\text{atoms}} \times 3$ `EFZ`

(1)

(2) $/|E| > 10^6$ eV$|F| > 10^4$ eV/Å

(3) $\Sigma_0(\mathcal{D})$(**??**)M_t

(4) $\mathcal{H}(\mathcal{D}) = \text{SHA-256}(\mathcal{D})$

(5) 80%10%10%

2Multi-Expert Training

Spring**??**

Algorithm 1 Spring

Require: $\mathcal{D}_{\text{train}}$ $\mathcal{D}_{\text{val}}$ $\mathcal{D}_{\text{calib}}$
Require: $T_{\max}$ $\varepsilon_{\text{conv}}$
Ensure: $V_{\text{consensus}}$
$\Psi_0 \leftarrow \infty$; $t \leftarrow 0$
$\mathcal{H}_{\text{data}} \leftarrow \text{SHA-256}(\mathcal{D}_{\text{train}})$
$\Sigma_0 \leftarrow \text{ComputeComplexity}(\mathcal{D}_{\text{train}})$ ▷
$M_0 \leftarrow \Xi(\mathcal{H}_{\text{data}}, \Sigma_0)$ ▷ M_t
while $t < T_{\max}$ **and** $\Psi_t > \varepsilon_{\text{conv}}$ **do**
  $\{\mathcal{D}_1, \ldots, \mathcal{D}_{M_t}\} \leftarrow \text{StratifiedBootstrap}(\mathcal{D}_{\text{train}}, M_t)$
  parallel for $m = 1$ **to** M_t **do**
  $\theta_m^{(t)} \leftarrow \text{TrainExpert}(\mathcal{D}_m, \theta_m^{\text{init}})$ ▷
  end parallel for
  $\mathbf{s}_{\text{YAJIE}}^{(t)} \leftarrow \text{YajieConsensus}(\{f_{\theta_m}\}_{m=1}^{M_t}, \mathcal{D}_{\text{calib}})$
  $\Psi_t \leftarrow \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_i \text{Var}_m[f_m(\mathbf{R}_i)]$
  $\mathcal{D}_{\text{train}} \leftarrow \text{RemoveNoise}(\mathcal{D}_{\text{train}}, \mathbf{s}_{\text{YAJIE}}^{(t)}, \tau_{\text{noise}})$
  $M_{t+1} \leftarrow \Xi(\mathcal{H}_{\text{data}}, \Psi_t, \mathbf{s}_{\text{YAJIE}}^{(t)}, \Sigma_0)$
  $t \leftarrow t + 1$
end while
$V_{\text{consensus}} \leftarrow \sum_m \omega_m f_{\theta_m}$ ▷
$S_{\text{CERCIS}} \leftarrow \text{CercisScore}(V_{\text{consensus}}, \mathbf{s}_{\text{YAJIE}}, \Psi_t, \mathcal{D}_{\text{val}})$
return $(V_{\text{consensus}}, \mathbf{s}_{\text{YAJIE}}, \Psi_t, M_t, S_{\text{CERCIS}})$

3YajieYajie Denoising

YAJIE[?]Spring

- $\mathbf{R}_i M_t$

$$s_{\text{YAJIE}}(\mathbf{R}_i) = \exp\left(-\frac{1}{M_t}\sum_{m=1}^{M_t}\left(\frac{\hat{E}_m(\mathbf{R}_i) - \hat{E}_{\text{YAJIE}}(\mathbf{R}_i)}{\sigma_{\text{calib},m}}\right)^2\right) \tag{1}$$

- τ_{noise}"/"
- 2

Remark 3.1. YAJIE——
textit
textit $M_t - 11$——

4$M_t M_t$ Binding

M_tSPRING MD""[?]

Definition 3.2 (M_t)**.** $tM_t^{auto}\mathcal{H}(\mathcal{D})Lyapunov\Psi_{t-1}$ $Yajie\mathbf{s}_{\text{YAJIE}}^{(t-1)}$

$$M_t^{auto} = \Xi(\mathcal{H}(\mathcal{D}); \Psi_{t-1}, \mathbf{s}_{\text{YAJIE}}^{(t-1)}, \Sigma_0(\mathcal{D})) \tag{2}$$

M_t—— ""M_t
textitco-emerge

Proposition 3.3 (M_t)**.** $\mathcal{DH}(\mathcal{D})\Sigma_0(\mathcal{D})$ M_t"$M_t = 8M = 1$"

5CercisCercis Scoring

CercisSpring MD[?]

$$S_{\text{Cercis}} = Q + \eta \cdot N \tag{3}$$

- $Q \in [0, 1]$YajieLyapunov

$$Q = \frac{1}{2}\left(\bar{s}_{\text{Yajie}} + \exp(-\Psi_T/\Psi_0)\right) \tag{4}$$

- $N \in [0, 1]$

$$N = \min\left(1, \frac{|\mathcal{D}_{\text{eff}}|}{|\mathcal{D}_{\text{train}}|} \cdot \frac{\sigma_E}{\Delta E_{\text{target}}}\right) \tag{5}$$

- $\eta = 0.2$——
$S_{\text{Cercis}} \in [0, 1.2]$1.2

3.3

`spring-md train`YAML

Listing 2: `spring_config.yaml`

```yaml
# Spring MD
train:
  data:
    path: "./data/train.xyz"
    format: "extxyz"
    train_ratio: 0.8
    val_ratio: 0.1
    calib_ratio: 0.1

  model:
    architecture: "nep"     # nep | deepmd | ace | mace
    num_neurons: [30, 30]   #
    cutoff: 6.0             #  (  )
    descriptor_dim: 128     #

  spring:
    M_min: 3                #
```

```
    M_max: 32               #
    noise_fraction: 0.05    #
    consensus_threshold: 0.7 # Yajie

  optim:
    epochs: 1000
    batch_size: 32
    learning_rate: 1.0e-3
    scheduler: "cosine"
    weight_decay: 1.0e-5

  output:
    dir: "./spring_output"
    save_checkpoints: true
    checkpoint_freq: 50
```

3.4

spring-md trainaudit_report.json

Listing 3: audit_report.json

```
{
  "spring_md_version": "1.0.0",
  "timestamp": "2026-07-01T12:00:00Z",
  "data_hash": "a3f8c2...",
  "complexity": {
    "Sigma_0": 1.42,
    "energy_entropy": 3.21,
    "energy_range": [-78.5, -72.3],
    "num_species": 2,
    "num_configurations": 10000
  },
  "training": {
    "M_t": 12,
    "M_t_history": [8, 12, 12, 12],
    "total_epochs": 450,
    "converged": true,
    "Lyapunov_final": 0.0012,
    "Lyapunov_initial": 0.0450
  },
  "yajie": {
```

```
    "mean_consensus": 0.92,
    "consensus_histogram": [0, 5, 23, 142, ...],
    "noise_fraction_removed": 0.032,
    "calibration_set_size": 1000
  },
  "cercis": {
    "score": 1.08,
    "quality_Q": 0.92,
    "coverage_N": 0.80,
    "discount_eta": 0.20
  }
}
```

4 spring-md run

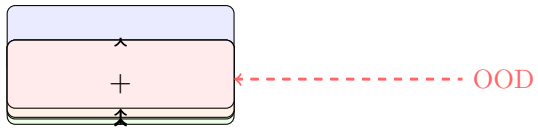
4.1

Listing 4: spring-md run

```
spring-md run [OPTIONS] <POTENTIAL> <INPUT_STRUCTURE>

INPUT:
  <POTENTIAL>           (potential.pt)
  <INPUT_STRUCTURE>     (XYZ, POSCAR, CIF, LAMMPS data)

OPTIONS:
  --config <FILE>      MD (YAML)
  --output <DIR>        (: ./md_output/)
  --engine <ENGINE>    MD: lammps | ase (: lammps)
  --ensemble <ENS>     : nve | nvt | npt (: nvt)
  --temperature <FLOAT> (K) (: 300)
  --timestep <FLOAT>    (fs) (: 0.5)
  --steps <INT>         (: 100000)
  --audit-every <INT>   (: 100)
  --device <STR>        (: cuda)

OUTPUT ( <DIR>/ ):
  trajectory.xyz        (extended XYZ)
  trajectory.h5         (HDF5, /)
  arbiter_report.json  Arbiter
  md_log.txt           MD
  thermo.dat            (T, P, E...)
```

4.2

spring-md run
textbfSpring MDMD——LAMMPSASE



ARBITER

Figure 3: `spring-md run`ARBITERMDk OODARBITERMD ——

4.3 Audit Hook

ARBITERMD`--audit-every`100

Protocol 4.1 (Arbiter). 1 $\mathbf{R}(t)$ MD

2 $M_t > 1$ ARBITER$M_t\mathbf{R}(t)$

- YAJIE$s_{\text{YAJIE}}(\mathbf{R}(t))$(**??**)
- $\sigma_E(t) = \text{std}_m[\hat{E}_m(\mathbf{R}(t))]$
- $\bar{\sigma}_F(t) = \frac{1}{N_{\text{atoms}}} \sum_i \text{std}_m[\hat{F}_{i,m}(\mathbf{R}(t))]$

3 **OOD** $\mathbf{R}(t)$ACEACE $\mathbf{R}(t)$Mahalanobis95%OOD

4 **Cercis**

$$s^{\text{frame}}_{\text{CERCIS}}(t) = s_{\text{YAJIE}}(\mathbf{R}(t)) \cdot (1 - \mathbb{1}_{\text{OOD}} \cdot 0.5) \tag{6}$$

$\mathbb{1}_{\text{OOD}}$OODOODCERCIS

5 $ts_{\text{YAJIE}}\sigma_E\bar{\sigma}_F$ OOD$s^{\text{frame}}_{\text{CERCIS}}$ARBITER

6 **OOD** n_{warn}OOD$n_{\text{warn}} = 10$ ARBITER`stderr`——
textit

Remark 4.2 (MD). ARBITEROOD ARBITER
textit
textitMDOOD ARBITER"OOD"——

4.4 MD

Listing 5: `md_config.yaml`

```
# Spring MD
run:
  engine: "lammps"          # lammps | ase
  ensemble: "nvt"           # nve | nvt | npt
  temperature: 300.0        #  (K)
```

```
  pressure: 1.0            #  (bar, NPT)
  timestep: 0.5            #  (fs)
  total_steps: 100000      #

  thermostat:
    type: "nose_hoover"    # nose_hoover | berendsen | langevin
    tau: 100.0             #  (fs)

  barostat:                # NPT
    type: "nose_hoover"
    tau: 1000.0

  audit:
    frequency: 100         #  ()
    ood_warn_consecutive: 10 # OOD
    save_full_experts: false #

  output:
    dir: "./md_output"
    format: "xyz"          # xyz | h5 | both
    thermo_freq: 10        #
```

4.5 LAMMPS

`--engine lammps`Spring MDLAMMPS

(1) PyTorchLibTorch C++TorchScript LAMMPS`pair_style`Spring MDLAMMPS `pair_style``pair_coeff`

(2) **LAMMPS** Spring MDPythonLAMMPS LAMMPSPython`lammps`

(3) Spring MDLAMMPS `audit-every` LAMMPS——Spring MD

(4) MD10× $\sim$ 50× M_t100M_t = 8 2%$\sim$5%——

4.6 ASE

`--engine ase`Spring MDASEAtomic Simulation EnvironmentPython API

(1) ASE`Calculator` `get_potential_energy()``get_forces()`

(2) ASE`Observer` Spring MD`ArbiterObserver``audit-every`

(3) ASELAMMPS——MD IO

5 spring-md audit

5.1

Listing 6: spring-md audit

```
spring-md audit [OPTIONS] <TRAJECTORY> [POTENTIAL]

INPUT:
  <TRAJECTORY>          (XYZ, H5, LAMMPS dump)
  [POTENTIAL]           (; )

OPTIONS:
  --config <FILE>       (YAML)
  --output <DIR>        (: ./audit_output/)
  --M <INT>             ()
  --ood-method <STR>    OOD: ace_mahalanobis | knn | isolation_forest
  --per-frame           (: 10)
  --reference <FILE>

OUTPUT ( <DIR>/ ):
  arbiter_report.json  Arbiter
  per_frame.csv
  ood_analysis.json    OOD
  summary.txt
```

5.2

spring-md audit

Table 2: spring-md audit

	- MD /
+	Yajie Cercis Spring MD OOD
+ A + B	A vs B Yajie

5.3

`spring-md audit` Spring MD$V_A V_A V_B$ DeepMDMACE

Protocol 5.1. 1 $\mathbf{R}(t) V_A V_B E_A(t), E_B(t) \mathbf{F}_A(t), \mathbf{F}_B(t)$

2 $\Delta E(t) = |E_A(t) - E_B(t)| \Delta F(t) = \|\mathbf{F}_A(t) - \mathbf{F}_B(t)\|$

3 ""——$\Delta E(t) > \varepsilon_E \Delta F(t) > \varepsilon_F$ $\varepsilon_E, \varepsilon_F$10 meV/atom0.1 eV/Å

4 ACE

5 "Yajie"——""

$$s^{\text{cross}}_{\text{Yajie}}(t) = \exp\left(-\frac{(E_A(t) - E_B(t))^2}{\sigma_A^2 + \sigma_B^2}\right) \tag{7}$$

6

5.4

1`spring-md audit`

-
-
-
-
- NVENVT

Remark 5.2. Arbiter
textitMD——
MD""

6 spring-md compareCercis

6.1

Listing 7: spring-md compare

```
spring-md compare [OPTIONS] <POTENTIALS...> --test-set <FILE>

INPUT:
  <POTENTIALS...>       (potential_1.pt potential_2.pt ...)
  --test-set <FILE>     (extended XYZ)

OPTIONS:
  --config <FILE>       (YAML)
  --output <DIR>        (: ./compare_output/)
  --metrics <LIST>     : energy, forces, stress, yajie, cercis
  --per-config          (: )
  --reference <FILE>    (; "")
  --plot                (matplotlib)

OUTPUT ( <DIR>/ ):
  ranking.json         Cercis
  ranking.csv           (CSV)
  per_config.csv        ( --per-config)
  comparison_report.pdf ( --plot)
```

6.2

spring-md compare

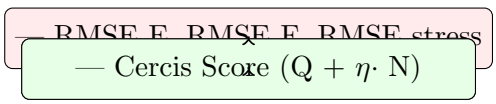


Figure 4: spring-md compareMD SCX//Cercis

6.3

Algorithm 2 Cercis

Require: K $\{V_k\}_{k=1}^{K}$ $\mathcal{D}_{\text{test}} = \{\mathbf{R}_i, E_i^{\text{ref}}, \mathbf{F}_i^{\text{ref}}\}_{i=1}^{N}$
Ensure: Cercis rank_k

for $k = 1$ **to** K **do**
  V_k $\mathcal{A}_k$
  //
  for $i = 1$ **to** N **do**
    $\hat{E}_i^{(k)} \leftarrow V_k(\mathbf{R}_i)$; $\hat{\mathbf{F}}_i^{(k)} \leftarrow -\nabla V_k(\mathbf{R}_i)$
  end for
  $\text{RMSE}_E^{(k)} \leftarrow \sqrt{\frac{1}{N}\sum_i (\hat{E}_i^{(k)} - E_i^{\text{ref}})^2}$
  $\text{RMSE}_F^{(k)} \leftarrow \sqrt{\frac{1}{3NN_{\text{atoms}}}\sum_i \|\hat{\mathbf{F}}_i^{(k)} - \mathbf{F}_i^{\text{ref}}\|^2}$
  //
  if $\mathcal{A}_k$ **then**
    YAJIE $\{s_{\text{YAJIE}}^{(k)}(\mathbf{R}_i)\}$
    $\bar{s}_{\text{YAJIE}}^{(k)} \leftarrow \frac{1}{N}\sum_i s_{\text{YAJIE}}^{(k)}(\mathbf{R}_i)$
    $f_{\text{OOD}}^{(k)} \leftarrow \text{OOD}$
  else
    $\bar{s}_{\text{YAJIE}}^{(k)} \leftarrow \text{NaN}$; $f_{\text{OOD}}^{(k)} \leftarrow \text{NaN}$
  end if
  // **Cercis**
  $S_{\text{CERCIS}}^{(k)} \leftarrow \text{CercisScore}(V_k, \bar{s}_{\text{YAJIE}}^{(k)}, \text{RMSE}_E^{(k)})$
end for
$S_{\text{CERCIS}} \rightarrow \text{rank}_k$
return $\{(\text{rank}_k, S_{\text{CERCIS}}^{(k)}, \text{RMSE}_E^{(k)}, \bar{s}_{\text{YAJIE}}^{(k)})\}_{k=1}^{K}$

6.4 Cercis

Listing 8: `ranking.json`

```
{
  "test_set": {
    "path": "./data/test.xyz",
    "num_configurations": 500,
    "energy_range": [-80.2, -71.5],
    "num_species": 2
  },
  "rankings": [
    {
      "rank": 1,
      "potential": "spring_nep_M12",
      "source": "spring-md train",
      "M_t": 12,
```

```
      "Cercis_score": 1.08,
      "RMSE_E": 0.85,
      "RMSE_F": 52.3,
      "Yajie_consensus": 0.92,
      "Lyapunov_final": 0.0012,
      "OOD_fraction": 0.01
    },
    {
      "rank": 2,
      "potential": "spring_nep_M8",
      "source": "spring-md train",
      "M_t": 8,
      "Cercis_score": 0.96,
      "RMSE_E": 0.92,
      "RMSE_F": 58.1,
      "Yajie_consensus": 0.87,
      "Lyapunov_final": 0.0024,
      "OOD_fraction": 0.03
    },
    {
      "rank": 3,
      "potential": "deepmd_v2",
      "source": "DeepMD-kit",
      "M_t": null,
      "Cercis_score": null,
      "RMSE_E": 0.88,
      "RMSE_F": 55.0,
      "Yajie_consensus": null,
      "OOD_fraction": null
    }
  ],
  "notes": ["DeepMDCercisYajie"]
}
```

Remark 6.1 (Spring). **NEP/DeepMD/MACE`spring-md compare` Cercis Spring MD——Spring MD””RMSE ”””””**

7 Arbiter

7.1

ArbiterSpring MDMD

1 **Audit Non-Separability** ArbiterTrajectory_hash SHA-256—— ""

2 **Per-Frame Traceability** MD""——
textit

3 **Human-Machine Dual-Readability** JSONnotes JSON Schemajq

7.2 Schema

Table 3: ArbiterJSON Schema

report_meta.version	string	Spring MD
report_meta.timestamp	string	ISO 8601
report_meta.report_type	string	"train" — "run" — "audit"
potential.path	string	
potential.architecture	string	"nep" — "deepmd" — "ace" — "mace"
potential.M_t	int	
potential.data_hash	string	SHA-256
potential.Cercis_score	float	Cercis
simulation.ensemble	string	"nve" — "nvt" — "npt"
simulation.temperature	float	K
simulation.timestep	float	fs
simulation.total_steps	int	
simulation.engine	string	MD"lammps" — "ase"
simulation.trajectory_hash	string	SHA-256
audit_frames[N]	array	
audit_frames[].step	int	MD
audit_frames[].time	float	ps
audit_frames[].Yajie_consensus	float	Yajie $s_{\text{Yajie}} \in [0,1]$
audit_frames[].energy_std	float	σ_E (meV/atom)
audit_frames[].force_std	float	$\bar{\sigma}_F$ (meV/Å)
audit_frames[].is_OOD	bool	OOD
audit_frames[].OOD_score	float	OODMahalanobis/
audit_frames[].Cercis_frame	float	Cercis
audit_frames[].flags	[string]	"low_consensus" — "high_dispersion" — "critical_ood"
summary.total_frames	int	
summary.mean_consensus	float	Yajie
summary.consensus_below_0.5	int	0.5
summary.OOD_fraction	float	OOD
summary.Cercis_mean	float	Cercis
summary.flags_summary	object	flag

7.3

ARBITERsummary

- **Healthy Trajectory** mean_consensus > 0.85OOD_fraction < 0.05 Cercis_mean > 0.80
- **Warning Trajectory** mean_consensus $\in [0.5, 0.85]$OOD_fraction $\in [0.05, 0.20]$ OOD
- **Unreliable Trajectory** mean_consensus < 0.5 OOD_fraction > 0.20 (a) (b) (c)

8 API

8.1

Spring MD

(1) **PyTorch**`potential.pt` `torch.save()`

- `state_dict`
- M_t`state_dict`
-
- PyTorch_`audit_metadata`

(2) **YAML**`potential.yaml`

- Cercis
- M_tYajieM_t
-
-

8.2 API

Listing 9: Python API

```
from spring_md import SpringPotential

#
potential = SpringPotential.load("spring_output/potential.pt")

#
energy = potential.predict_energy(positions, cell, atomic_numbers)
forces = potential.predict_forces(positions, cell, atomic_numbers)

#
expert_energies = potential.predict_energy_all_experts(positions, cell,
    atomic_numbers)

#
meta = potential.audit_metadata
print(f"M_t = {meta['M_t']}")
print(f"Cercis = {meta['Cercis_score']}")
print(f"Yajie consensus = {meta['Yajie_mean_consensus']}")
```

8.3

Spring MD

Table 4:

Extended XYZ	`.xyz`	✓	✓	//
HDF5	`.h5, .hdf5`	✓	✓	
LAMMPS dump	`.dump, .lammpstrj`	✓		LAMMPS
ASE trajectory	`.traj`	✓	✓	ASE
NetCDF	`.nc`	✓		AMBER/CHARMM

8.4 ArbiterAPI

Listing 10: Python API

```
from spring_md import ArbiterReport

#
report = ArbiterReport.load("md_output/arbiter_report.json")

#
print(report.summary())
# => "Healthy: 99.2% consensus, 1.3% OOD, Cercis 0.89"

# OOD
ood_frames = report.get_ood_frames(threshold=0.5)
for frame in ood_frames:
    print(f"Step {frame.step}: OOD score = {frame.OOD_score:.2f}")

#
low_consensus = report.get_frames_by_flag("low_consensus")

#
report.plot_consensus_over_time(save="consensus_plot.png")

# CSV
report.export_csv("arbiter_report.csv")
```

9 LAMMPS/ASE

9.1

Spring MDLAMMPSASE
textbfTransparent Audit Injection

(1) Spring MDLAMMPSASE

(2) Spring MDLAMMPSASE/——LAMMPS/ASE Spring MD

(3) MD

9.2 LAMMPS

Listing 11: LAMMPSPython

```python
import subprocess
from spring_md import SpringPotential, ArbiterHook

# 1. LAMMPS
potential = SpringPotential.load("potential.pt")
potential.export_lammps("spring_potential.so") # TorchScript  LAMMPS pair style

# 2. LAMMPS
lammps_script = f"""
units         metal
atom_style    atomic
pair_style    spring_md
pair_coeff    * * spring_potential.so {element_list}

read_data     input.data
velocity      all create {temp} {seed}
fix           nvt all nvt temp {temp} {temp} {tau}
thermo        {thermo_freq}
thermo_style  custom step temp pe ke etotal press
dump          traj all xyz {dump_freq} trajectory.xyz
run           {total_steps}
"""

with open("in.spring", "w") as f:
    f.write(lammps_script)

# 3. LAMMPS
```

```
proc = subprocess.Popen(
    ["lmp", "-in", "in.spring"],
    stdout=subprocess.PIPE, stderr=subprocess.PIPE
)

# 4. Arbiter
hook = ArbiterHook(
    potential=potential,
    trajectory_file="trajectory.xyz",
    audit_every=100,
    output="arbiter_report.json"
)
hook.attach_to_process(proc) # ktrajectory.xyz

proc.wait()
hook.finalize() #
```

9.3 ASE

ASEASEPython

Listing 12: ASEPython

```
from ase.io import read
from ase.md.nvt import NVTBerendsen
from ase import units
from spring_md import SpringPotential, ArbiterObserver

# 1. ASE Calculator
potential = SpringPotential.load("potential.pt")
atoms = read("input.xyz")
atoms.calc = potential.to_ase_calculator()

# 2. MD
dyn = NVTBerendsen(
    atoms,
    timestep=0.5 * units.fs,
    temperature=300 * units.kB,
    taut=100 * units.fs
)

# 3. Arbiter
```

```
observer = ArbiterObserver(
    potential=potential,
    audit_every=100,
    output="arbiter_report.json"
)
dyn.attach(observer, interval=100)

# 4. MD
trajectory_writer = Trajectory("trajectory.traj", "w", atoms)
dyn.attach(trajectory_writer.write, interval=10)
dyn.run(100000)

# 5.
observer.save_report()
```

9.4

Table 5: Spring MDMD

	LAMMPS	**ASE**	
NVE	✓	✓	
NVT	✓	✓	
NPT	✓	✓	
	✓	✓	M_t
	✓	✓	
OOD	✓	✓	ACE
	✓	✓	JSON
Langevin	✓	✓	
Nose-Hoover	✓	✓	
	✓	✓	
GPU	✓	✓	PyTorch

10

10.1 1DFT

——DFTMD

```
# ================================================================
# 1DFT
# ================================================================
# DFTVASP/CP2K/Quantum ESPRESSOextended XYZ
# data/dft_train.xyz (10000)

# ================================================================
# 2
# ================================================================
spring-md train data/dft_train.xyz \
    --config configs/train.yaml \
    --arch nep \
    --output ./outputs/Si_potential \
    --device cuda

#  ./outputs/Si_potential/
#   potential.pt
#   potential.yaml
#   audit_report.json       Cercis + Yajie + Mt
#   training_log.csv

# ================================================================
# 3
# ================================================================
cat ./outputs/Si_potential/potential.yaml
#
#   Cercis_score: 1.08
#   M_t: 12
#   Yajie_mean_consensus: 0.92
#   Lyapunov_final: 0.0012
#   Data_hash: a3f8c2e1...

# ================================================================
# 4MDArbiter
# ================================================================
spring-md run ./outputs/Si_potential/potential.pt data/initial_Si.xyz \
    --config configs/md.yaml \
```

```
    --engine lammps \
    --ensemble nvt \
    --temperature 300 \
    --steps 100000 \
    --audit-every 100 \
    --output ./outputs/Si_md_run

#  ./outputs/Si_md_run/
#   trajectory.xyz
#   arbiter_report.json     Arbiter
#   thermo.dat
#   md_log.txt              MD

# ================================================================
# 5
# ================================================================
spring-md audit ./outputs/Si_md_run/trajectory.xyz \
    ./outputs/Si_potential/potential.pt \
    --output ./outputs/Si_md_audit

# Python API
python -c "
from spring_md import ArbiterReport
report = ArbiterReport.load('./outputs/Si_md_run/arbiter_report.json')
print(report.summary())
report.plot_consensus_over_time('consensus.png')
ood = report.get_ood_frames()
print(f'OOD frames: {len(ood)}/{report.summary_data[\"total_frames\"]}')
"

# ================================================================
# 6
# ================================================================
spring-md compare \
    ./outputs/Si_potential/potential.pt \
    ./outputs/Si_potential_alt/potential.pt \
    ./external/deepmd_Si.pb \
    --test-set data/test_Si.xyz \
    --output ./outputs/Si_comparison

#
```

```
cat ./outputs/Si_comparison/ranking.json
```

10.2 2

LAMMPSSpring MD

```
# LAMMPS dumpDeepMD
# Spring MD

# 1
spring-md audit legacy_run/dump.lammpstrj \
    --output ./audit_legacy

#

# 2Spring
# Spring
spring-md audit legacy_run/dump.lammpstrj \
    ./outputs/Si_potential/potential.pt \
    --output ./audit_legacy_cross

#   Spring   vs DeepMD
#
```

10.3 3

```
#!/bin/bash
#

MATERIALS=("Si" "SiO2" "SiC" "Si3N4" "Al2O3" "MgO" "TiN" "ZrO2")

for mat in "${MATERIALS[@]}"; do
    echo "=== Processing $mat ==="

    #
    spring-md train data/${mat}_dft.xyz \
        --output ./ht_outputs/${mat}_potential \
        --config configs/ht_train.yaml

    # Cercis
    CERCIS=$(python -c "
```

```
import json
with open('./ht_outputs/${mat}_potential/audit_report.json') as f:
    print(json.load(f)['cercis']['score'])
")

    # Cercis > 0.7MD
    if (( $(echo "$CERCIS > 0.7" | bc -l) )); then
        spring-md run ./ht_outputs/${mat}_potential/potential.pt \
            data/${mat}_initial.xyz \
            --output ./ht_outputs/${mat}_md \
            --config configs/ht_md.yaml
    else
        echo "Skipping $mat: Cercis score $CERCIS < 0.7"
    fi
done

#
spring-md compare ./ht_outputs/*_potential/potential.pt \
    --test-set data/ht_test.xyz \
    --output ./ht_outputs/final_ranking
```

11 NEP/DeepMD/MACE

11.1

Table 6: Spring MDMD

	NEP	**DeepMD**	**ACE**	**MACE**	**Spring MD**
	✓	✓	✓	✓	✓ ()
=?	55	55	55	55	✓
$M_t > 1$	55	55	55	55	✓
Yajie	55	55	55	55	✓
Cercis	55	55	55	55	✓
LAMMPS	✓	✓		55	✓
ASE	✓	✓	✓	✓	✓
Arbiter	55	55	55	55	✓
OOD	55	55	55	55	✓
	55	55	55	55	✓ (Cercis)
	55	55	55	55	✓
	55	55	55	55	✓
	55	55	55	55	✓
	55	55	55	55	✓ (SHA-256)

11.2

Definition 11.1. *MD*
textbf

(1) M_tCercisYajie

(2)

(3)

(4)

(5)

??MDNEPDeepMDACEMACEGAPMTP **0/5**Spring MD

Remark 11.2 (——). 0/5Spring MD5/5

11.3

Spring MDNEP/DeepMD/ACE

Table 7: NEP1×

NEP ($M = 1$)	1×	1
DeepMD ($M = 1$)	2-3×	1
ACE ($M = 1$)	0.5×	1
MACE ($M = 1$)	3-5×	1
Spring MD ($M_t = 8$)	6-8×	1 + 8 +
Spring MD ($M_t = 12$)	9-12×	1 + 12 +

Remark 11.3 (-). $M_t = 8$Spring MDNEP6-84-7×
textbf——M_t/Yajie "MD"——NEP/DeepMD "MD"—— Spring MD

12

12.1 Spring MD

Spring MD

1 Yajie0.95
textit Schrödinger —— DFT
textit
textit

2 M_t = 86-8> 10^5 > 10^5 Spring MDGPU——

3 **MD** 2%-5% > 10^6> 10^7 M_t> 16

4 **Spring** NEP/DeepMD/MACESpring MD"" `spring-md runspring-md compare` YajieCercis—— Spring MD
textit`spring-md train`

5 NEPGPUMDDeepMDDeePMD-kitMACEMACE-MP—— Spring MD

12.2

Spring MD

- MD Arbiter
- `spring-md compare`CercisRMSE
- MD `spring-md audit`
- – Cercis3

Spring MD
textbf

- NEPMACE3-8×
- **ReaxFF** Spring MD NEPDeepMDACEMACE
- Spring MD

12.3 SCX

Spring MDSCX[?, ?, ?, ?, ?] SCX

- **SCX** ——$M = 1M > 1$ -

- **Spring**[?] —— $\rightarrow\rightarrow\rightarrow\rightarrow$
- **Spring Trainer**[?] —— YajieM_tCercis
- **Spring MD** —— ArbiterLAMMPS/ASE

12.4

(1) Spring MD ""on-the-fly learning——ArbiterOOD DFTActive Learning

(2) GPUSpring $M_t = 32$

(3) Web MDCercisOOD

(4) Spring MD DFT

(5) MDArbiter MD——MD

13

Spring MD——"" Spring MD`trainrunauditcompare` DFTMD

Spring MD

(1) **BORN AUDITED** `spring-md train`M_tCercis Yajie—— ""

(2) `spring-md run`""" + Arbiter" CercisYajieOOD——
textitMD

(3) Spring MD`spring-md train`NEP/DeepMD/MACE

`spring-md auditspring-md compare`

Spring MD"NEP""DeepMD"——
textit MD"-" Spring MD"-"—— NEP/DeepMD/MACE "auditable by design"

Spring MDv1.0SCX Spring—— SCX1SCX2SCX3- SpringYajie "auditable MD"

References

[1] SCX, "SCX," *SCX* — , 2026.

[2] SCX, "SCX," *SCX* — , 2026.

[3] SCX, "Yajie," *SCX* — , 2026.

[4] SCX, "Spring," *SCX* — , 2026.

[5] SCX, "Spring," *SCX* — , 2026.

[6] Z. Fan *et al.*, "Neuroevolution machine learning potentials: Combining high accuracy and low cost in atomistic simulations," *Phys. Rev. B*, vol. 104, p. 104309, 2021.

[7] L. Zhang, J. Han, H. Wang, R. Car, and W. E, "Deep Potential Molecular Dynamics: A Scalable Model with the Accuracy of Quantum Mechanics," *Phys. Rev. Lett.*, vol. 120, p. 143001, 2018.

[8] R. Drautz, "Atomic cluster expansion for accurate and transferable interatomic potentials," *Phys. Rev. B*, vol. 99, p. 014104, 2019.

[9] I. Batatia, D. P. Kovács, G. N. C. Simm, C. Ortner, and G. Csányi, "MACE: Higher Order Equivariant Message Passing Neural Networks for Fast and Accurate Force Fields," *Advances in Neural Information Processing Systems*, 2022.

[10] A. P. Bartók, M. C. Payne, R. Kondor, and G. Csányi, "Gaussian Approximation Potentials: The Accuracy of Quantum Mechanics, without the Electrons," *Phys. Rev. Lett.*, vol. 104, p. 136403, 2010.

[11] S. Plimpton, "Fast Parallel Algorithms for Short-Range Molecular Dynamics," *J. Comp. Phys.*, vol. 117, pp. 1–19, 1995.

[12] A. H. Larsen *et al.*, "The Atomic Simulation Environment—a Python library for working with atoms," *J. Phys.: Condens. Matter*, vol. 29, p. 273002, 2017.

A ArbiterJSON Schema

ArbiterJSON SchemaJSON Schema Draft 2020-12

Listing 13: ArbiterJSON Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Arbiter Audit Report",
  "type": "object",
  "required": ["report_meta", "potential", "simulation", "audit_frames", "summary"],
  "properties": {
    "report_meta": {
      "type": "object",
      "required": ["version", "timestamp", "report_type"],
      "properties": {
        "version": {"type": "string", "pattern": "^\\d+\\.\\d+\\.\\d+$"},
        "timestamp": {"type": "string", "format": "date-time"},
        "report_type": {"type": "string", "enum": ["train", "run", "audit"]}
      }
    },
    "potential": {
      "type": "object",
      "required": ["path", "architecture", "M_t", "data_hash"],
      "properties": {
        "path": {"type": "string"},
        "architecture": {"type": "string"},
        "M_t": {"type": "integer", "minimum": 2},
        "data_hash": {"type": "string", "pattern": "^[a-f0-9]{64}$"},
        "Cercis_score": {"type": "number", "minimum": 0, "maximum": 1.2},
        "Yajie_mean_consensus": {"type": "number", "minimum": 0, "maximum": 1},
        "Lyapunov_final": {"type": "number", "minimum": 0}
      }
    },
    "simulation": {
      "type": "object",
      "required": ["ensemble", "temperature", "timestep", "total_steps"],
      "properties": {
        "ensemble": {"type": "string", "enum": ["nve", "nvt", "npt"]},
        "temperature": {"type": "number", "minimum": 0},
        "timestep": {"type": "number", "minimum": 0},
        "total_steps": {"type": "integer", "minimum": 1},
        "engine": {"type": "string", "enum": ["lammps", "ase"]},
        "trajectory_hash": {"type": "string"}
      }
    },
    "audit_frames": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["step", "time", "Yajie_consensus", "Cercis_frame"],
        "properties": {
          "step": {"type": "integer", "minimum": 0},
          "time": {"type": "number"},
          "Yajie_consensus": {"type": "number", "minimum": 0, "maximum": 1},
          "energy_std": {"type": "number", "minimum": 0},
          "force_std": {"type": "number", "minimum": 0},
          "is_OOD": {"type": "boolean"},
```

```
        "OOD_score": {"type": "number", "minimum": 0},
        "Cercis_frame": {"type": "number", "minimum": 0, "maximum": 1},
        "flags": {"type": "array", "items": {"type": "string"}}
       }
      }
    },
    "summary": {
      "type": "object",
      "required": ["total_frames", "mean_consensus"],
      "properties": {
        "total_frames": {"type": "integer"},
        "mean_consensus": {"type": "number", "minimum": 0, "maximum": 1},
        "OOD_fraction": {"type": "number", "minimum": 0, "maximum": 1},
        "Cercis_mean": {"type": "number", "minimum": 0, "maximum": 1},
        "flags_summary": {"type": "object"}
      }
    }
  }
}
```

B

spring-md

```
spring-md <COMMAND> [OPTIONS]

COMMANDS:
  train     (Training)
  run       (MD Simulation)
  audit     (Trajectory Auditing)
  compare   (Potential Comparison)
  help

GLOBAL OPTIONS:
  --version
  --help
```

Table 8: Spring MD

0	
1	
2	Lyapunov $\Psi_T > \varepsilon_{\text{conv}}$
3	MD
4	
5	GPU/CUDA

C

Spring MD

```
# pip
pip install spring-md

#
# - Python >= 3.9
# - PyTorch >= 2.0 (with CUDA support for GPU)
# - LAMMPS (, lammps)
# - ASE >= 3.22 (, ase)

#
spring-md --version
# Spring MD v1.0.0 (SCX Audit Middleware)
# PyTorch backend: CUDA 12.1
# Engines: lammps (found), ase (v3.23.0)
```